



جامعة ٦ أكتوبر
October 6 University



علوم الحاسب ونظم المعلومات
Info Sys & Comp Science

TTL Exfiltration Monitor

Done by:

Yousef Amr Elmenshawy –23014943

Yousef Samer Abdelkader –23016427

Yousef Fathallah Gaafar –23011767

Yousef Ashraf Shazly –23013123

Yassen Assem Elmasry –23013518

Moustafa Ashraf Mohamed –23012710

Supervised by:

Dr. Germien Gergessedhom

Supervised by:

Eng. Ibrahim Muhammed

Eng. Tasnim Gamal

1. Overview & Objectives

This project implements a network monitoring tool designed to detect suspicious packets based on anomalously low Time-To-Live(TTL) values. The tool automatically starts an Nmap scan against the suspected source IP to gather further intelligence.

Objectives:

- Monitor all inbound IP traffic using raw sockets
- Flag any packet with TTL below 10
- Display alert with timestamp, source/destination IP, protocol, and TTL
- Automatically run an Nmap scan on the suspicious IPs
- Suppress duplicate alerts using a 10-second cooldown
- Support both Windows and Linux without code modification

2. Background & Technical Concepts

- **TTL(Time-To-Live)**

TTL is an 8-bit field located in the IP packet header. Its primary purpose is to prevent packets from circulating indefinitely across networks. Every router that forwards a packet decrements the TTL value by one; when the value reaches zero, the packet is discarded and an ICMP message is returned to the sender. Different operating systems initialize TTL with different default values –Windows uses 128, while Linux and macOS use 64. Under normal network conditions, a packet arriving at its destination retains most of its original TTL value. Therefore, a packet observed with a TTL below 10 is unusual – it either originated from a host dozen or more hops away, or its TTL was set to a low value, both of which need investigation.

- **Why low TTL is Suspicious?**

Low TTL values typically indicate either traceroute activity – where packets are sent with TTLs starting from 1 to map network paths – or covert techniques like tunneling and data exfiltration that manipulate TTL to avoid detection. In either case, a source IP consistently sending low-TTL packets is a red flag worth investigating.

- **Raw Sockets**

Standard sockets operate at the application layer. Raw sockets bypass this and give direct access to IP-level data, allowing the tool to read every packet passing through the interface – not just packets addressed to the host.

- **Nmap**

A widely-used open-source network scanner. The `-sV` flag detects service versions running on open ports; `-T4` sets an aggressive timing template for faster results.

3. System Architecture

The tool is structured as a single Python script divided into five functions, each responsible for a distinct task. The table below shows the role of each function

Function	Role
Check_permission()	Validates root/admin privileges before any operation
Get_local_ip()	Detects the host IP address for Windows socket binding
Parse_ip_header()	Unpacks raw bytes to extract TTL, protocol, source and destination IP
Sniff_packets()	Main capture loop – handles alerting and triggers Nmap scans
Nmap_scan()	Launches an Nmap subprocess against flagged source IPs

Raw packets are captured from the network interface, the IP header is parsed to extract the TTL and addresses, packets below the threshold trigger a console alert, and previously unseen source IPs are passed to the Nmap module for deeper investigation.

4. Code Walkthrough

Go function by function and write one short paragraph per function explaining the logic, not just what it does:

- **check_permissions():** Explain that raw sockets are a privileged operation. On Windows, `ctypes.windll.shell32.IsUserAnAdmin()` checks administration. On Linux, `os.geteuid() == 0` checks for root. The tool exits immediately if permissions are insufficient to avoid cryptic socket errors later.
- **get_local_ip():** Explain the trick — it creates a UDP socket and calls `connect()` to Google's DNS (8.8.8.8) without actually sending any data. The OS populates the local address as a side effect, which is then read with `getsockname()`.
- **parse_ip_header():** Highlight the `struct.unpack('!BBHHBHH4s4s', data[:20])` line. The `!` means big-endian (network byte order). The format unpacks the 20-byte IPv4 header — TTL is at index 5, protocol at index 6, source IP at index 8. `socket.inet_ntoa()` converts the 4-byte binary address to a readable dotted string.
- **sniff_packets():** This is the core loop. Explain the 14-byte offset on Linux (`raw_data[14:]`) skips the Ethernet frame header to reach the IP header. The `alert_cooldown` dictionary suppresses repeated alerts for the same IP within 10 seconds. The `already_scanned` set ensures Nmap only runs once per unique IP per session.
- **nmap_scan():** Explain that `subprocess.run()` launches Nmap as a child process, captures stdout, and prints it. The `timeout=60` kills the scan if it hangs. The `FileNotFoundError` catch handles systems where Nmap isn't installed gracefully.

5. Testing & Results

- **Test Environment**
 - **Operating System:** Linux (Ubuntu 22.04)
 - **Python Version:** 3.12.3
 - **Network:** Local LAN (192.168.77.124)
 - **Tools:** Nmap installed, root privileges active

- **Test Case 1**

To simulate suspicious low-TTL traffic, a ping was sent from a second machine on the network with a manually forced TTL value of 5 using the following command:

`ping --ttl 5 192.168.1.1`

The tool successfully detected the packet and printed the following alert to the console:

```
[!] [23:42:58] LOW TTL DETECTED
    SRC:192.168.64.1 -> DST:239.255.255.250
    TTL:1 | Protocol:17
[>] Launching Nmap scan on 192.168.64.1 ...
```

The alert correctly displayed the source IP, destination IP, TTL value, and protocol number. This confirms that the packet capture and header parsing are working as expected.

- **Test Case 2**

To test the alert suppression logic, the same low-TTL ping was sent repeatedly from the same source IP within a 10-second window. The tool printed only one alert for the first packet and suppressed all subsequent alerts from that IP until the cooldown period expired. After 10 seconds, a new alert was correctly generated upon the next detection. This confirms that the cooldown dictionary is functioning as intended and prevents console flooding during sustained low-TTL traffic.

- **Test Case 3**

Upon the first detection of a new source IP, the tool automatically launched an Nmap scan using the configured `-sV -T4` flags. A sample of the Nmap output produced was:

```
[>] Launching Nmap scan on 192.168.64.1 ...
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-12 23:42 EEST
Nmap scan report for 192.168.64.1
Host is up (0.00030s latency).
Not shown: 993 closed tcp ports (reset)
PORT      STATE SERVICE          VERSION
135/tcp    open  msrpc            Microsoft Windows RPC
139/tcp    open  netbios-ssn     Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds?
902/tcp    open  ssl/vmware-auth  VMware Authentication Daemon 1.10 (Uses VNC, SOAP)
912/tcp    open  vmware-auth     VMware Authentication Daemon 1.0 (Uses VNC, SOAP)
2179/tcp   open  vmrdp?
7070/tcp   open  ssl/realserver?
MAC Address: 00:15:5D:35:E7:88 (Microsoft)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 35.59 seconds
```

The scan successfully identified open ports and running services on the flagged host.

6. Security & Ethical Considerations

The tool requires root/Administrator privileges, meaning it must only be deployed on networks the operator owns or has explicit written permission to monitor. Running it on unauthorized or shared networks is itself a security violation, as it exposes all traffic passing through the interface. Additionally, the automated Nmap scanning feature carries legal risk — actively scanning an IP without permission may violate laws such as the Computer Fraud and Abuse Act (CFAA) in the US or similar legislation elsewhere. In a real deployment, scanned IPs should be validated against the local subnet before any active scanning is triggered, and external IPs should be logged for manual review rather than scanned automatically.